

Reviewer Pack — Phase Cell Count Bounded by Tropical Proof Rank

Entry 5f34041e6974 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-30 17:57:06 UTC

Phase Cell Count Bounded by Tropical Proof Rank

Entry ID: 5f34041e6974 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-30 17:57:06 UTC

1. Conjecture

Field A (mathematical branch): Tropical Circuit Weight Analysis (BOUNDED_ARITHMETIC) **Field B** (complexity object): Phase Space Complexity

Statement:

The number of phase cells in the phase space of a tropical circuit encoding a bounded arithmetic proof line is at most the Tropical Proof Rank of that proof.

Rationale (proposer's reasoning):

This tests A1 (Phase Stability Axiom), which asserts that ID_o -total functions have polynomially many phase cells under scaling. If the Tropical Proof Rank bounds phase cell counts, it aligns with A1's polynomial bound and directly connects phase complexity to proof strength.

Taxonomy category: BOUNDED_ARITHMETIC (status at proposal time:)

Framework membership: framework fw_a1a152ae17, role: elaboration

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 3fdd6a52b49b08c4

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations

def tropical_zero():
    return float('-inf')

def tropical_add(a, b):
    if a == tropical_zero() or b == tropical_zero():
        return max(a, b)
    else:
        return max(a + b, 0)

def tropical_mul(a, b):
    if a == tropical_zero() or b == tropical_zero():
        return tropical_zero()
    else:
        return a + b

def gaussian_elimination(A):
    n = len(A)
    m = len(A[0])
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        for k in range(m):
            A[i][k] /= A[i][i]
        for j in range(n):
            if j != i:
                factor = A[j][i]
                for k in range(m):
                    A[j][k] -= factor * A[i][k]
    return A

def tropical_rank(A):
    n = len(A)
    m = len(A[0])
```

```

rank = 0
for i in range(n):
    if all(abs(A[j][i]) == tropical_zero() for j in range(n)):
        continue
    rank += 1
return rank

def generate_tropical_circuit(n, m):
    A = [[random.choice([tropical_zero(), random.randint(-10, 10)]) for _ in range(m)] for _ in range(n)]
    B = [random.choice([tropical_zero(), random.randint(-10, 10)]) for _ in range(m)]
    return A, B

def phase_cells(A, B):
    n = len(A)
    m = len(B)
    cells = set()
    for i in range(2**n):
        x = [int(bit) for bit in f"{i:0{n}b}"]
        y = sum(x[j] * A[j][k] for j, k in enumerate(range(m))) + B
        cell = tuple(sorted(y))
        cells.add(cell)
    return len(cells)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []
    for n in n_values:
        A, B = generate_tropical_circuit(n, n)
        tropical_rank_value = tropical_rank(A)
        phase_cell_count = phase_cells(A, B)
        results.append({
            "n": n,
            "tropical_rank_value": tropical_rank_value,
            "phase_cell_count": phase_cell_count
        })
    metric_name = "Phase Cell Count"
    metric_value = sum(result["phase_cell_count"] for result in results) / len(results)
    instances_tested = len(results)
    conjecture_holds = all(result["phase_cell_count"] <= result["tropical_rank_value"] for result in results)
    counterexample = "" if conjecture_holds else "mapping_undefined"
    return {
        "metric_name": metric_name,
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

```

```

results.append(result)

mean_value = sum(result["metric_value"] for result in results) / len(results)
std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not result["conjecture_holds"] for result in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e9234095.py", line 107, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e9234095.py", line 83, in run_trial
    phase_cell_count = phase_cells(A, B)
                        ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e9234095.py", line 71, in phase_cells
    y = sum(x[j] * A[j][k] for j, k in enumerate(range(m))) + B
          ~~~~~^~~~~~
TypeError: unsupported operand type(s) for +: 'float' and 'list'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with `TypeError` before producing results, preventing evaluation of conjecture validity | next: Debug the `phase_cells` function's type error in the tropical circuit implementation

11. Audit log (LLM calls)

Total LLM calls: 6

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	preregistration	ollama_remote	qwen3:8b	0	0	24121
2	novelty	ollama_remote	qwen3:8b	0	0	20650
3	novelty	ollama_remote	qwen3:8b	0	0	13600
4	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18252
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11789
6	judge	ollama_remote	qwen3:8b	0	0	15879

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 104292 ms total latency. Provider mix: {'ollama_remote': 6}

(full prompt+response transcripts available in `research/audit/5f34041e6974.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5f34041e6974.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/5f34041e6974.tar.gz` (if generated)